

Universidad Autónoma de Tamaulipas

Facultad de Ingeniería Tampico



ASIGNATURA

PROGRAMACION DE INTERFACES Y PUERTOS

6to. Semestre – Grupo “I” 2025 -1

TRABAJO

Desarrollo de Prácticas y Proyectos

UNIDAD

3 - PROGRAMACIÓN DE INTERFACES

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Izaguirre Cortes Emanuel	60%
Turrubiates Mejia Gilberto	20%
García Salas Yahir Misael	20%
Total:	100%

Índice

Índice.....	1
Repositorio(s) de Prácticas	2
P1. Arduino _Binario	2
Descripción de la practica	2
Introducción.....	2
Componentes para el desarrollo de la practica	3
Desarrollo	3
Conclusiones.....	6
P2. Arduino _ASCII	7
Descripción de la practica	7
Introducción.....	7
Componentes para el desarrollo de la practica	7
Desarrollo	8
Conclusiones.....	14
P3. Arduino _	14
Descripción de la practica	14
Introducción.....	15
Componentes para el desarrollo de la practica	15
Desarrollo	15
Conclusiones.....	18
P4. Arduino _	18
Descripción de la practica	18
Introducción.....	19
Componentes para el desarrollo de la practica	19
Desarrollo	19
Conclusiones.....	22
Fuentes consultadas (Si aplica).....	23

Repositorio(s) de Prácticas

Practica	Repositorio
Practica 1 a la 4	https://github.com/Emanuel-Izaguirre-Cortes-03/Pip_2025_1_eq_4_i/tree/main/Practicas/Practicas_U3

P1. Arduino_Binario

Descripción de la practica

Esta práctica utiliza 8 LEDs conectados a los pines digitales del Arduino (del 5 al 12) para representar un número decimal en su equivalente binario. Al enviar un número a través del monitor serial (por ejemplo, 150), el programa lo convierte a binario (10010110) y enciende los LEDs correspondientes a los bits en 1.

Cada bit del número se evalúa usando divisiones sucesivas (módulo) para obtener si es 1 o 0, comenzando por el bit más significativo (128) hasta el menos significativo (1). Luego, con la función digitalWrite, se encienden los LEDs que representan los bits en 1.

El número mostrado permanece encendido durante 4.5 segundos, luego todos los LEDs se apagan, y el sistema queda listo para recibir otro número.

Si el número ingresado es mayor a 255 o no es válido, se interpreta como un número fuera de rango y se encienden **todos los LEDs** como advertencia.

Introducción

En el mundo de la electrónica digital y la programación, el sistema binario es fundamental, ya que todos los datos procesados por computadoras y microcontroladores se representan mediante ceros y unos. En esta práctica, se utiliza una placa Arduino para visualizar la conversión de números decimales a binarios mediante una serie de ocho LEDs. Cada LED representa un bit, lo que permite al usuario observar de forma física y clara cómo un número decimal se transforma en su equivalente binario.

Mediante la comunicación serial, el usuario puede ingresar un número entre 0 y 255 desde el monitor serial del IDE de Arduino. El programa procesa este número, lo descompone en bits, y enciende los LEDs correspondientes según su valor binario. Esta actividad refuerza conceptos básicos de lógica digital, uso de puertos de salida y manejo de entradas por

comunicación serial, siendo una base útil para proyectos más avanzados en electrónica y programación de microcontroladores.

Componentes para el desarrollo de la practica

- 1 placa Arduino (UNO, Nano, etc.)
- 8 LEDs
- 8 resistencias (220 ohm recomendadas)
- Cables de conexión
- Protoboard
- Computadora con el IDE de Arduino

Desarrollo

Para llevar a cabo esta práctica, se realizó una conexión de 8 LEDs a los pines digitales del Arduino (del 5 al 12), utilizando resistencias para proteger los componentes de posibles sobrecargas. Cada LED representa un bit del número binario, desde el bit más significativo (128) hasta el menos significativo (1).

En el entorno de desarrollo de Arduino (IDE), se escribió un programa que inicia la comunicación serial a 9600 baudios. El programa espera que el usuario ingrese un número decimal desde el monitor serial. Una vez recibido, este número se convierte a su equivalente binario mediante operaciones de división y módulo, y cada bit se almacena en una variable (b128, b64, ..., b1).

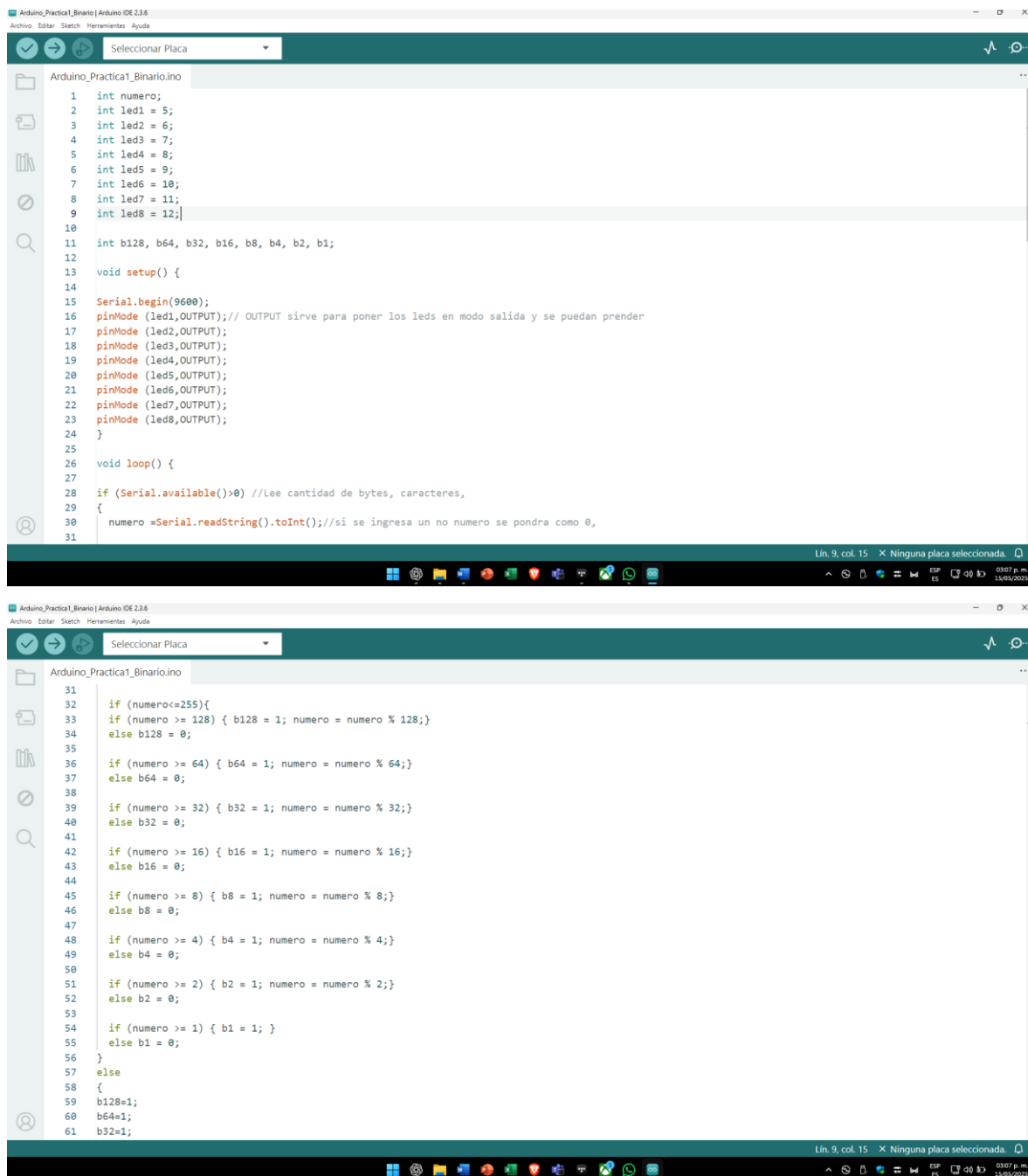
Dependiendo del valor de cada bit, se enciende o apaga el LED correspondiente utilizando la función `digitalWrite()`. Si el número recibido es válido (entre 0 y 255), se representa correctamente. En caso contrario, como medida de advertencia, se encienden todos los LEDs indicando que el valor está fuera de rango o no fue válido.

El patrón de encendido se mantiene visible durante 4.5 segundos, tiempo en el cual el usuario puede visualizar la representación binaria. Posteriormente, todos los LEDs se apagan, y el sistema queda listo para recibir un nuevo número.

Este proceso se repite continuamente dentro del bucle `loop()`, lo que permite al usuario interactuar con el sistema cuantas veces sea necesario.

Durante la implementación se tuvo cuidado de respetar la polaridad de los LEDs, así como verificar la correcta asignación de pines tanto en el circuito como en el código. También se hicieron pruebas con diferentes números para comprobar que la conversión binaria se realizaba correctamente.

Codigo



```

1  int numero;
2  int led1 = 5;
3  int led2 = 6;
4  int led3 = 7;
5  int led4 = 8;
6  int led5 = 9;
7  int led6 = 10;
8  int led7 = 11;
9  int led8 = 12;
10
11 int b128, b64, b32, b16, b8, b4, b2, b1;
12
13 void setup() {
14
15   Serial.begin(9600);
16   pinMode (led1,OUTPUT); // OUTPUT sirve para poner los leds en modo salida y se puedan prender
17   pinMode (led2,OUTPUT);
18   pinMode (led3,OUTPUT);
19   pinMode (led4,OUTPUT);
20   pinMode (led5,OUTPUT);
21   pinMode (led6,OUTPUT);
22   pinMode (led7,OUTPUT);
23   pinMode (led8,OUTPUT);
24 }
25
26 void loop() {
27
28   if (Serial.available() > 0) //Lee cantidad de bytes, caracteres,
29   {
30     numero = Serial.readString().toInt(); //si se ingresa un no numero se pondra como 0,
31
32     if (numero <= 255) {
33       if (numero >= 128) { b128 = 1; numero = numero % 128; }
34       else b128 = 0;
35
36       if (numero >= 64) { b64 = 1; numero = numero % 64; }
37       else b64 = 0;
38
39       if (numero >= 32) { b32 = 1; numero = numero % 32; }
40       else b32 = 0;
41
42       if (numero >= 16) { b16 = 1; numero = numero % 16; }
43       else b16 = 0;
44
45       if (numero >= 8) { b8 = 1; numero = numero % 8; }
46       else b8 = 0;
47
48       if (numero >= 4) { b4 = 1; numero = numero % 4; }
49       else b4 = 0;
50
51       if (numero >= 2) { b2 = 1; numero = numero % 2; }
52       else b2 = 0;
53
54       if (numero >= 1) { b1 = 1; }
55       else b1 = 0;
56     }
57     else
58     {
59       b128=1;
60       b64=1;
61       b32=1;

```

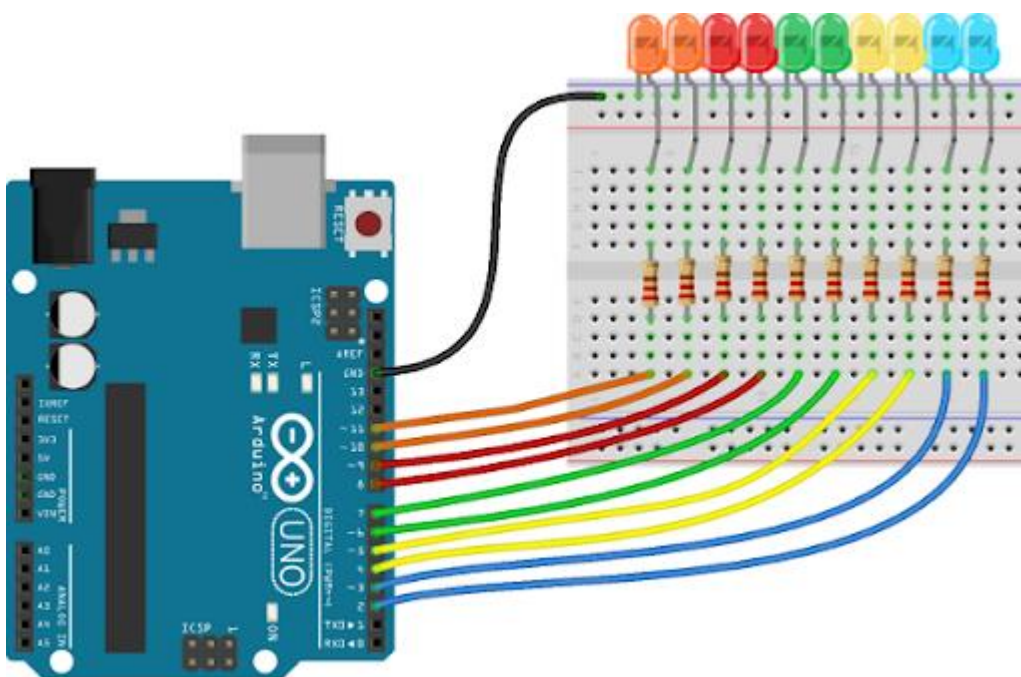
```

Arduino, Practica1_Binario | Arduino IDE 2.3.6
Archivo Editor Sketch Herramientas Ayuda
Seleccionar Placa

Arduino, Practica1_Binario.ino
59 b128=1;
60 b64=1;
61 b32=1;
62 b16=1;
63 b8=1;
64 b4=1;
65 b2=1;
66 b1=1;
67 }
68 digitalWrite(led1,b128);
69 digitalWrite(led2,b64);
70 digitalWrite(led3,b32);
71 digitalWrite(led4,b16);
72 digitalWrite(led5,b8);
73 digitalWrite(led6,b4);
74 digitalWrite(led7,b2);
75 digitalWrite(led8,b1);
76
77 delay(4500);
78 digitalWrite(led1,0);
79 digitalWrite(led2,0);
80 digitalWrite(led3,0);
81 digitalWrite(led4,0);
82 digitalWrite(led5,0);
83 digitalWrite(led6,0);
84 digitalWrite(led7,0);
85 digitalWrite(led8,0);
86
87 delay(250);
88
89

```

Lin. 9, col. 15 X Ninguna placa seleccionada. 03:07 p. m. 15/05/2023



Conclusiones

1. **Comprensión del sistema binario:**

Esta práctica permitió reforzar el conocimiento sobre la conversión de números decimales a binarios, mostrando de forma visual cómo cada bit se representa físicamente mediante el encendido de un LED.

2. **Manejo de salidas digitales en Arduino:**

Se aprendió a utilizar pines digitales del Arduino como salidas para controlar dispositivos (en este caso, LEDs), aplicando funciones básicas como `pinMode()` y `digitalWrite()`.

3. **Interacción con el monitor serial:**

Se practicó la lectura de datos desde el monitor serial, comprendiendo cómo recibir entradas del usuario para que el programa tome decisiones con base en esos datos.

4. **Desarrollo de lógica de programación:**

La práctica incluyó la construcción de una lógica que convierte números en binario usando operaciones aritméticas, fortaleciendo la lógica condicional y la manipulación de variables.

5. **Importancia del diseño de hardware:**

Se valoró la importancia de usar resistencias para proteger los componentes, así como el orden y claridad en las conexiones sobre la protoboard.

En resumen, esta práctica integró conceptos de electrónica digital y programación básica de Arduino, sirviendo como base para proyectos más complejos donde se necesite representar o interpretar datos en diferentes formatos.

P2. Arduino_ASCII

Descripción de la practica

En esta práctica se desarrolló un sistema con Arduino UNO que permite visualizar en binario los valores ASCII de los caracteres ingresados desde el monitor serial. El sistema utiliza ocho LEDs conectados a pines digitales del Arduino, los cuales representan los 8 bits de un byte. Cuando el usuario escribe una palabra o frase en el monitor serial, el programa analiza cada carácter, obtiene su valor ASCII y lo convierte a su equivalente binario. Posteriormente, cada bit se refleja mediante el encendido (1) o apagado (0) de los LEDs, permitiendo al usuario ver gráficamente el código binario correspondiente a cada letra. Esta práctica fortalece el entendimiento de la codificación binaria, los códigos ASCII y el manejo de cadenas y estructuras repetitivas en programación con Arduino.

Introducción

Esta práctica tiene como objetivo representar visualmente los valores binarios de los caracteres ingresados desde el monitor serial, utilizando un sistema de 8 LEDs conectados a un Arduino UNO. Al introducir una palabra o frase, el programa convierte cada letra a su valor en el sistema ASCII, y posteriormente a binario. Cada bit se refleja con el encendido o apagado de los LEDs. Esta actividad fortalece los conocimientos sobre el sistema binario, la codificación ASCII y el control de salidas digitales en Arduino, así como el uso de estructuras como ciclos for y cadenas de texto en programación.

Componentes para el desarrollo de la practica

- 1 placa Arduino (UNO, Nano, etc.)
- 8 LEDs
- 8 resistencias (220 ohm recomendadas)
- Cables de conexión
- Protoboard
- Computadora con el IDE de Arduino

Desarrollo

1. Conecta los componentes según el diagrama de conexiones. Asegúrate de que las polaridades de los LEDs y las conexiones a los pines del Arduino sean correctas.
2. Abre el IDE de Arduino en tu computadora.
3. Copia y pega el código anterior en una nueva ventana del IDE.
4. Selecciona la placa "Arduino UNO" en el menú "Herramientas" > "Placa".
5. Selecciona el puerto serial correcto al que está conectado tu Arduino en el menú "Herramientas" > "Puerto".
6. Sube el código al Arduino haciendo clic en el botón "Subir" (la flecha hacia la derecha).
7. Una vez que la carga se haya completado, abre el "Monitor Serial" haciendo clic en el icono del pequeño rectángulo en la esquina superior derecha del IDE (o presionando Ctrl+Shift+M).
8. En la ventana del Monitor Serial, asegúrate de que la velocidad de baudios esté configurada en 9600.
9. Escribe una palabra o frase en el cuadro de texto superior del Monitor Serial y haz clic en "Enviar" o presiona Enter.
10. Observa cómo los LEDs del Arduino se encienden y apagan para representar el código binario de cada carácter que envías. El valor ASCII y su representación binaria también se mostrarán en el Monitor Serial.

Análisis y Resultados:

Al ingresar caracteres en el Monitor Serial, el programa Arduino procesa cada carácter individualmente. Para cada carácter:

- Se obtiene su valor numérico correspondiente en la tabla ASCII.
- Este valor ASCII (que es un número entero entre 0 y 255 para caracteres ASCII estándar) se convierte a su representación binaria de 8 bits.
- Cada bit de esta representación binaria se refleja en el estado de un LED: un LED encendido representa un '1' y un LED apagado representa un '0'. El orden de los LEDs corresponde al orden de los bits, generalmente con el LED conectado al pin digital más bajo representando el bit menos significativo (LSB) y el LED conectado al pin digital más alto representando el bit más significativo (MSB).

Al observar la secuencia de encendido y apagado de los LEDs para cada carácter, se puede visualizar directamente su código binario. La información mostrada en el Monitor Serial complementa esta visualización, permitiendo verificar la correcta conversión y representación

Codigo

```
int numero;  
int led1 = 5;  
int led2 = 6;  
int led3 = 7;  
int led4 = 8;  
int led5 = 9;  
int led6 = 10;  
int led7 = 11;  
int led8 = 12;  
  
int b128, b64, b32, b16, b8, b4, b2, b1; // Mayusculas 65 - 90 , minusculas  
97 - 122  
  
void setup() {
```

```
Serial.begin(9600);  
  
pinMode (led1,OUTPUT); // OUTPUT sirve para poner los leds en modo salida y se  
puedan prender  
pinMode (led2,OUTPUT);  
pinMode (led3,OUTPUT);  
pinMode (led4,OUTPUT);  
pinMode (led5,OUTPUT);  
pinMode (led6,OUTPUT);  
pinMode (led7,OUTPUT);  
pinMode (led8,OUTPUT);  
}  
  
void loop() {  
  
if (Serial.available()>0) // Si hay algo ingresado, cuando no esta vacio  
{  
    String palabra = Serial.readString(); //Agarra lo encontrado y lo convierte  
a String  
    palabra.trim();  
    for (int i = 0; i < palabra.length(); i++) //Recorre cada letra  
    {  
        numero = palabra[i]; // palabra[i] es un char y al intentar guardarlo en  
una variable int, se guardara su numero ascii  
  
        if (numero<=255)  
        {  
            if (numero >= 128) { b128 = 1; numero = numero % 128;}  
            else b128 = 0;  
            if (numero >= 64) { b64 = 1; numero = numero % 64;}  
            else b64 = 0;  
            if (numero >= 32) { b32 = 1; numero = numero % 32;}  
            else b32 = 0;
```

```
if (numero >= 16) { b16 = 1; numero = numero % 16;}
else b16 = 0;
if (numero >= 8) { b8 = 1; numero = numero % 8;}
else b8 = 0;
if (numero >= 4) { b4 = 1; numero = numero % 4;}
else b4 = 0;
if (numero >= 2) { b2 = 1; numero = numero % 2;}
else b2 = 0;
if (numero >= 1) { b1 = 1; }
else b1 = 0;
}
else
{
b128=1;
b64=1;
b32=1;
b16=1;
b8=1;
b4=1;
b2=1;
b1=1;
}

//Encender
digitalWrite(led1,b128);
digitalWrite(led2,b64);
digitalWrite(led3,b32);
digitalWrite(led4,b16);
digitalWrite(led5,b8);
digitalWrite(led6,b4);
digitalWrite(led7,b2);
digitalWrite(led8,b1);
// Apagar
delay(4500);
```

```
digitalWrite(led1,0);
digitalWrite(led2,0);
digitalWrite(led3,0);
digitalWrite(led4,0);
digitalWrite(led5,0);
digitalWrite(led6,0);
digitalWrite(led7,0);
digitalWrite(led8,0);
delay(250);
parpadeo(4, 500); // 4 segundos
delay(250);

} // Fin for que recorre cada letra
Serial.print("La palabra fue: ");
Serial.println(palabra);
} // Fin if

} // Fin loop

// Metodos
void parpadeo(int veces, int tiempo)
{ //Metodo de ayuda
for (int i = 0; i < veces; i++)
{

digitalWrite(led1, 1);
digitalWrite(led2, 1);
digitalWrite(led3, 1);
digitalWrite(led4, 1);
digitalWrite(led5, 1);
digitalWrite(led6, 1);
digitalWrite(led7, 1);
digitalWrite(led8, 1);
```

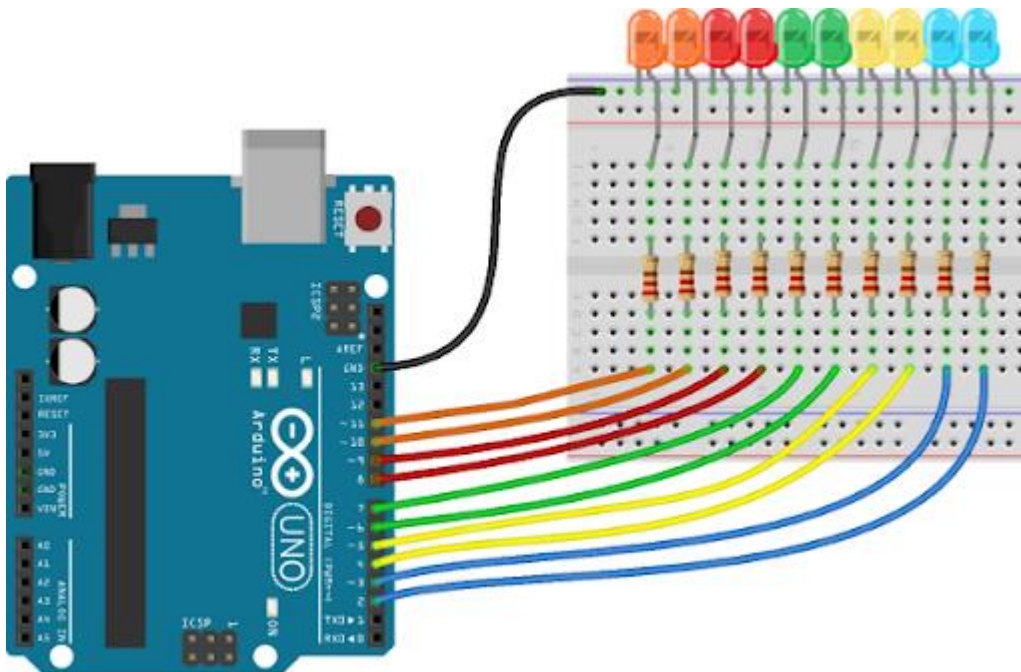
```

delay(tiempo);

digitalWrite(led1, 0);
digitalWrite(led2, 0);
digitalWrite(led3, 0);
digitalWrite(led4, 0);
digitalWrite(led5, 0);
digitalWrite(led6, 0);
digitalWrite(led7, 0);
digitalWrite(led8, 0);

delay(tiempo);
}
}

```



Conclusiones

Esta práctica demuestra de manera visual y práctica cómo los caracteres que utilizamos en la comunicación digital se representan internamente mediante códigos binarios, específicamente utilizando el estándar ASCII en este caso. Permite comprender la relación directa entre un carácter, su valor ASCII decimal y su equivalente binario. Además, refuerza los conceptos de manejo de entradas seriales, manipulación de bits y control de salidas digitales en la programación con Arduino. La utilización de estructuras repetitivas (como el bucle for) para la manipulación de los LEDs y el desplazamiento de bits ($\gg$ y $\&$) son aspectos clave aprendidos en esta actividad.

P3. Arduino_

Descripción de la practica

Esta práctica consiste en la lectura de los valores analógicos de cuatro sensores conectados a los pines analógicos A0, A1, A2 y A3 de una placa Arduino. Los valores leídos de cada sensor se concatenan en una cadena de texto, separados por el carácter "@", y luego se envían a través del puerto serial. Adicionalmente, la práctica implementa un control básico de un LED conectado al pin digital 13. El estado del LED (encendido o apagado) se controla mediante un valor numérico (0 o 1) recibido a través del monitor serial. Esta práctica introduce conceptos fundamentales como la lectura de sensores analógicos, el manejo de arreglos, la manipulación de cadenas de texto y la comunicación serial bidireccional con Arduino.

Introducción

En el mundo de la electrónica y la automatización, la capacidad de interactuar con el entorno a través de sensores es fundamental. Arduino, como plataforma de prototipado electrónico de código abierto, facilita la conexión y lectura de diversos tipos de sensores. Esta práctica explora la lectura simultánea de múltiples sensores analógicos, demostrando cómo obtener información del entorno y transmitirla para su posterior procesamiento o visualización. Además, se introduce un mecanismo básico de control actuando sobre un componente externo (un LED) a través de comandos enviados serialmente, sentando las bases para sistemas interactivos más complejos.

Componentes para el desarrollo de la practica

- Placa Arduino (UNO o similar)
- Cuatro sensores analógicos (LDR, potenciómetro, LM35, u otros que proporcionen una salida de voltaje variable)
- Cables de conexión (jumpers)
- Un LED
- Una resistencia de $220\ \Omega$ (para proteger el LED)
- Computadora con el IDE de Arduino instalado
- Cable USB A-B para conectar el Arduino a la computadora

Desarrollo

- **Configuración de Pines:** En la sección `setup()`, se configura el pin digital 13 como una salida (`pinMode(led, OUTPUT)`) para controlar el LED. Se inicializa la comunicación serial a una velocidad de 9600 baudios (`Serial.begin(9600)`) para permitir la transmisión y recepción de datos con la computadora. La función `Serial.setTimeout(100)` establece un tiempo de espera máximo de 100 milisegundos para la lectura de datos seriales.
- **Lectura de Sensores Analógicos:** Dentro del bucle principal `loop()`, se declara una variable de tipo `String` llamada `cadena` para almacenar los valores de los sensores. Se utiliza

un bucle `for` para iterar a través del arreglo `sensores`. En cada iteración, se lee el valor analógico del sensor conectado al pin especificado en el arreglo `sensores` utilizando la función `analogRead()`. El valor leído (que estará en un rango de 0 a 1023 para la mayoría de las placas Arduino) se almacena en el arreglo `valores_Sensores` en la posición correspondiente.

- **Construcción de la Cadena de Datos:** Después de leer cada sensor, el valor numérico se convierte a una cadena de texto utilizando `String()` y se concatena a la variable `cadena`, añadiendo un carácter "@" como separador entre los valores de los diferentes sensores.
- **Envío de Datos Seriales:** Una vez que se han leído los cuatro sensores y se ha construido la cadena, esta se envía a través del puerto serial utilizando `Serial.println(cadena)`. Esto permite visualizar los valores de los sensores en el monitor serial de la IDE de Arduino.
- **Recepción y Control Serial:** Se verifica si hay datos disponibles en el puerto serial utilizando `Serial.available()`. Si hay datos, se lee la cadena recibida con `Serial.readString()` y se intenta convertirla a un valor entero utilizando `toInt()`. Este valor entero se almacena en la variable `v`. Finalmente, se utiliza `digitalWrite(led, v)` para establecer el estado del LED. Si `v` es 1, el LED se encenderá; si es 0, se apagará. Es importante destacar que este código espera recibir un único valor entero para controlar el LED.
- **Retardo:** Se introduce un pequeño retardo de 100 milisegundos (`delay(100)`) para controlar la velocidad del bucle y evitar la lectura excesiva de los sensores.

Codigo

```
int v;
int sensores []= {A0,A1,A2,A3}; //Sensores digitales o analogicos
//LDR, potenciómetro, HCSR04, PIR, LM35, ...
int valores_Sensores []= {0, 0, 0,0};

int led = 13;

void setup() {
  pinMode(led, OUTPUT);
  Serial.begin(9600);
}
```


Conclusiones

Esta práctica demuestra la capacidad de Arduino para interactuar con múltiples sensores analógicos simultáneamente, procesar los datos leídos y transmitirlos a través de la comunicación serial. La utilización de arreglos simplifica la gestión de múltiples sensores, y la manipulación de cadenas permite la transmisión estructurada de información. Además, se introduce un mecanismo básico de control de un actuador (el LED) a través de comandos seriales, lo que ilustra la bidireccionalidad de la comunicación con Arduino. Esta experiencia sienta las bases para el desarrollo de proyectos más complejos que impliquen la adquisición de datos de diversos sensores y el control de dispositivos externos en respuesta a esos datos o a comandos remotos. Para una implementación más robusta del control del LED, se podría considerar la validación de la entrada serial para asegurar que solo se reciban los valores esperados (0 o 1).

P4. Arduino_

Descripción de la practica

Esta práctica se centra en la lectura de tres sensores analógicos conectados a los pines A0, A1 y A2 de una placa Arduino, utilizando funciones separadas para cada sensor. Los valores leídos se almacenan en un arreglo y se concatenan en una cadena de texto, separados por el carácter "@", para su envío a través del puerto serial. Adicionalmente, se implementa un control básico de un LED conectado al pin digital 13, cuyo estado (encendido o apagado) se determina por un valor numérico (0 o 1) recibido desde el monitor serial. Esta práctica refuerza conceptos de lectura de sensores analógicos, modularización del código mediante funciones, manejo de arreglos y comunicación serial bidireccional.

Introducción

La modularización del código es una práctica fundamental en programación, especialmente en proyectos de mayor envergadura. Esta práctica ilustra cómo organizar la lectura de múltiples sensores analógicos en funciones independientes, mejorando la legibilidad y el mantenimiento del código. Al igual que en la práctica anterior, se explora la lectura de sensores analógicos y la transmisión de datos a través del puerto serial, pero con un enfoque en la organización del código mediante funciones. Además, se mantiene el control de un LED a través de comandos seriales, demostrando la capacidad de Arduino para interactuar con el entorno y recibir instrucciones externas.

Componentes para el desarrollo de la practica

- Placa Arduino (UNO o similar)
- Tres sensores analógicos (LDR, LM35, PIR u otros con salida de voltaje variable)
- Cables de conexión (jumpers)
- Un LED
- Una resistencia de 220 Ω (para proteger el LED)
- Computadora con el IDE de Arduino instalado
- Cable USB A-B para conectar el Arduino a la computadora

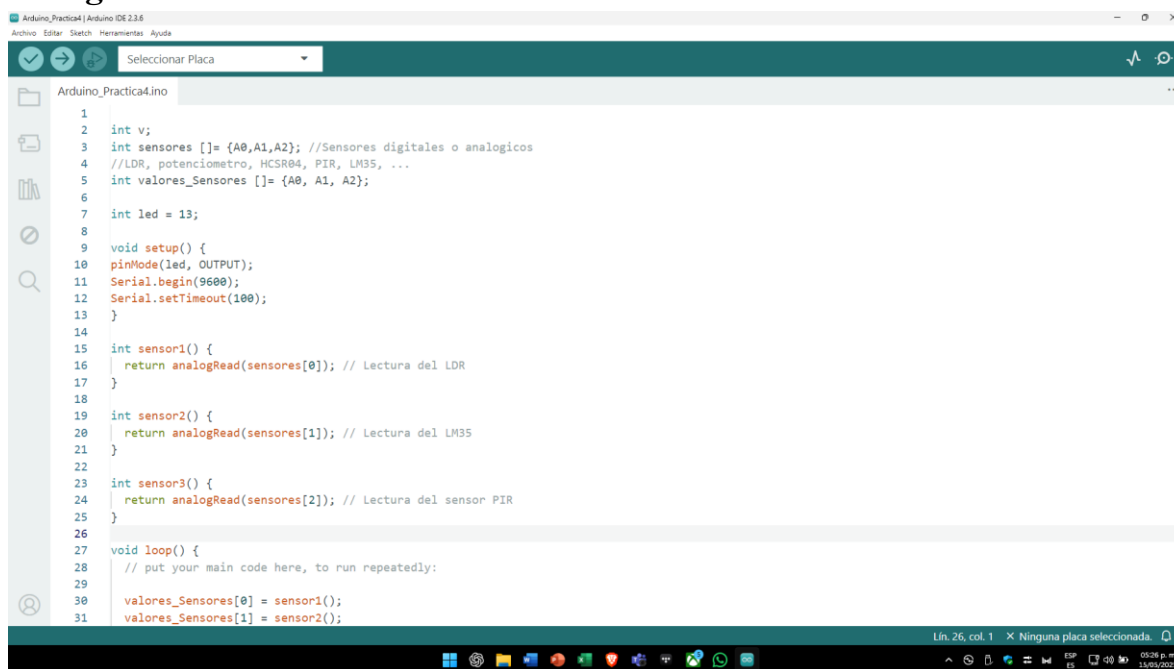
Desarrollo

- **Configuración de Pines:** En la sección `setup()`, se configura el pin digital 13 como una salida (`pinMode(led, OUTPUT)`) para controlar el LED. Se inicializa la comunicación serial a 9600 baudios (`Serial.begin(9600)`) y se establece un tiempo de espera de 100 milisegundos para la lectura serial (`Serial.setTimeout(100)`).
- **Funciones para la Lectura de Sensores:** Se definen tres funciones separadas: `sensor1()`, `sensor2()` y `sensor3()`. Cada función lee el valor analógico de un sensor específico conectado a los pines A0, A1 y A2, respectivamente, utilizando `analogRead()` y devuelve el valor leído.
- **Lectura de Sensores en el Bucle Principal:** En el bucle `loop()`, se llama a cada una de las funciones `sensor1()`, `sensor2()` y `sensor3()` para leer los

valores de los sensores. Los valores leídos se almacenan en el arreglo valores_Sensores en las posiciones correspondientes.

- **Construcción de la Cadena de Datos:** Se crea una variable cadena de tipo String para almacenar los valores de los sensores. Se utiliza un bucle for para iterar a través del arreglo valores_Sensores. En cada iteración, el valor numérico del sensor se convierte a una cadena con String() y se concatena a la variable cadena, añadiendo un "@" como separador.
- **Envío de Datos Seriales:** La cadena cadena, que contiene los valores de los tres sensores separados por "@", se envía a través del puerto serial utilizando Serial.println(cadena).
- **Recepción y Control Serial:** Se verifica si hay datos disponibles en el puerto serial con Serial.available(). Si hay datos, se lee la cadena recibida con Serial.readString() y se convierte a un entero con toInt(), almacenándolo en la variable v. Este valor v se utiliza para controlar el estado del LED mediante digitalWrite(led, v). Si v es 1, el LED se enciende; si es 0, se apaga.
- **Retardo:** Se introduce un retardo de 100 milisegundos (delay(100)) para controlar la velocidad del bucle.

Código



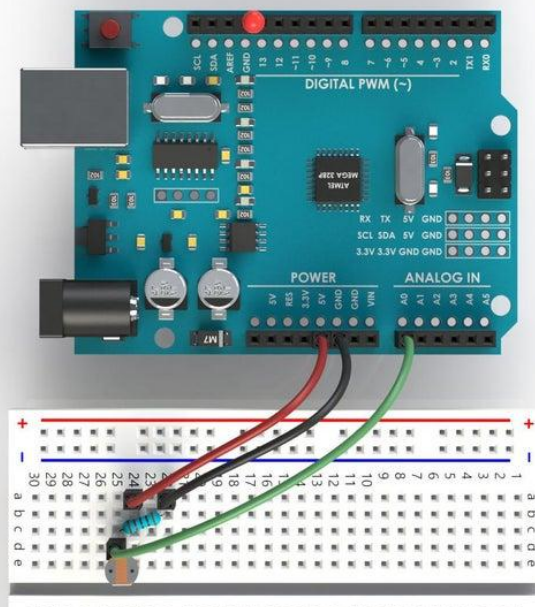
```

1  int v;
2
3  int sensores []= {A0,A1,A2}; //Sensores digitales o analogicos
4  //LDR, potenciómetro, HCSR04, PIR, LM35, ...
5  int valores_Sensores []= {A0, A1, A2};
6
7  int led = 13;
8
9  void setup() {
10     pinMode(led, OUTPUT);
11     Serial.begin(9600);
12     Serial.setTimeout(100);
13 }
14
15 int sensor1() {
16     return analogRead(sensores[0]); // Lectura del LDR
17 }
18
19 int sensor2() {
20     return analogRead(sensores[1]); // Lectura del LM35
21 }
22
23 int sensor3() {
24     return analogRead(sensores[2]); // Lectura del sensor PIR
25 }
26
27 void loop() {
28     // put your main code here, to run repeatedly:
29
30     valores_Sensores[0] = sensor1();
31     valores_Sensores[1] = sensor2();
  
```

```

Arduino_Practica4.ino
21 }
22
23 int sensor3() {
24     return analogRead(sensores[2]); // Lectura del sensor PIR
25 }
26
27 void loop() {
28     // put your main code here, to run repeatedly:
29
30     valores_Sensores[0] = sensor1();
31     valores_Sensores[1] = sensor2();
32     valores_Sensores[2] = sensor3();
33
34     String cadena = "";
35     for (int i = 0; i<3; i++)
36     {
37         // valores_Sensores[i] = analogRead(sensores[i]); // ya no se usa
38         cadena += String(valores_Sensores[i]) + "@";
39     }
40
41     Serial.println(cadena);
42
43     if(Serial.available()>0)
44     {
45         v = Serial.readString().toInt();
46         digitalWrite(led, v);
47     }
48
49     delay(100);
50 }
51

```



Conclusiones

Esta práctica demuestra la importancia de la modularización del código al organizar la lectura de sensores en funciones separadas, lo que facilita la lectura y el mantenimiento del código. Se refuerzan los conceptos de lectura de sensores analógicos, manejo de arreglos y comunicación serial bidireccional. La capacidad de controlar un actuador (el LED) a través de comandos seriales, combinada con la lectura de múltiples sensores, sienta las bases para sistemas interactivos más complejos. Para una mayor robustez, se podría implementar la validación de la entrada serial para asegurar que solo se reciban los valores esperados (0 o 1) para el control del LED.

Fuentes consultadas **(Si aplica)**

1. Arduino. (s.f.). *Referencia oficial de Arduino*. Recuperado de: <https://www.arduino.cc/reference/es>
2. Fritzing. (s.f.). *Software de diseño de circuitos electrónicos*. Recuperado de: <https://fritzing.org>
3. Tinkercad. (s.f.). *Simulador de circuitos y diseño 3D en línea*. Recuperado de: <https://www.tinkercad.com>
4. García, F. (2020). *Aprende Arduino desde cero*. España: Marcombo.
5. YouTube – Proyectos con Arduino. (s.f.). *Conversión de números decimales a binarios con LEDs*. Recuperado de: <https://www.youtube.com>